Upgrading Late-Stage Exploitation in PPO

Eric Sun
McGill University - 261103545

Sumukh Patel
McGill University - 261276753

Adrien Bélanger
McGill University - 261118168

Abstract

This project aims to reimplement the paper that introduced Proximal Policy Algorithms (PPO) and target three different modifications aimed at improving late-game performance on the Atari57 benchmark Schulman et al. [2017]. The first change is to introduce value function annealing to the algorithm, as it has been shown to do better exploitation in late-game usage Andrychowicz et al. [2021]. The second is to introduce a Long Short Term Memory (LSTM) model to better leverage the long-term dependencies Ni et al. [2022]. Thirdly, we introduce a replay buffer to try and improve the experience usage and gathering for improving game results in the 100 last episodes Wang et al. [2017].

1 Introduction

Policy gradient methods have been widely used in reinforcement learning. Schulman et al. introduced in 2017 PPO algorithms as a simple, stable and (eventually) widely-adopted on-policy algorithm Schulman et al. [2017]. PPO has been used in Reinforcement Learning from Human Feedback (RLHF) and complex games Ouyang et al. [2022], Berner et al. [2019]. In the original PPO paper, late-stage exploitation on the Atari benchmark (Figure 1), late stage rewards are lacking when compared to ACER, while still having overall higher avg. episode reward.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	**30**	0
(2) avg. episode reward over last 100 episodes	1	**28**	19	1

Figure 1: Table 2 from the original PPO paper, presenting the number of games won, averaged over three trials

The main objective of this project is thus: can targeted modifications to PPO diminish lack in performance in the late game training stage? We chose three modifications as our main focus: value function coefficient annealing inspired by insights from Andrychowicz et al. Andrychowicz et al. [2021], LSTM-based architecture to better capture long-term temporal dependencies Ni et al. [2022] and a replay buffer augmentation for better sample reuse Wang et al. [2017].

To evaluate performance, we chose the following 6 games: `Gopher`, `Video Pinball`, `UpNDown`, `Centipede`, `DemonAttack`, `BeamRider` and `Venture` (see annex for original paper scores) as they presented the most drastic difference in performance (with PPO not in first place) between the winning score and PPO's score.

We ran the benchmarks with the parameters from the original paper (see Annex 3).

2 Background

3 Methodology

3.1 General Code structure

The code is organized into five main parts. `game.py` takes care of creating the Atari environments via ALE/Gymnasium v5 and validates game names against the Atari 57 game list from the original paper Schulman et al. [2017]. `config.py` creates dataclasses (`EnvConfig`, `ActorCriticConfig`, `PPOConfig`, `BenchmarkConfig`) so that all hyperparameters are declared in one place. `model.py` contains the `ActorCritic` network and the `PPOModel` training loop, while `agent.py` provides a `PPOAgent` whose `train` method creates a model and then then takes care of rollout. Each experimental variant is implemented as a subclass of `PPOAgent`/`PPOModel`, overriding only the parts that change for their respective target modification. This was implemented to make it as easy to read for an external user, and as modular as possible so you could add models in the future.

Next, the observation Pipeline. We configure with `frameskip=1` and `repeat_action_probability=0.25` all as in the original paper. We also do `MaxAndSkipObservation(4)`, which pools over frames and produces a 4× frame skip (as in the original paper). We use the `ResizeObservation(84×84)`; `GrayscaleObservation`; and `FrameStackObservation(4)`.

As in the benchmarks results leading to the annexed paper, benchmarking for the project is performed over 3 random seeds per game. Episode returns are saved at the end of the episode and plotted as learning curves over game frames for better comparison with the original paper's results. The final reported score is the mean episode return over the last 100 episodes, again like table 6 from the PPO paper Schulman et al. [2017].

3.2 Base PPO

Our loss function follows the original paper Schulman et al. [2017]:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta]$$

where $\varepsilon = 0.1$, $c_1 = 1.0$, $c_2 = 0.01$, and both the learning rate (lr) and clipping parameter are being annealed (see annex 3 for full parameters).

The actor-critic setup follows Mnih et al. Mnih et al. [2015] (like the PPO paper does Schulman et al. [2017]): two convolutional layers with separate heads for the actor and the critic.

Training collects 128 steps across 8 parallel actors for each update. Then, it computes GAE advantages. Then, it runs 3 epochs of minibatch SGD with Adam optimization.

3.3 PPO with value-function annealing

In base PPO, our c_1 param is fixed throughout training. Andrychowicz et al. Andrychowicz et al. [2021] showed that the value loss weighting significantly affects on-policy performance. Thus, we implement this change in hopes of getting better late-stage performance. We decay c_1 from 1.0 to 0.1 over the full training run:

$$c_1(t) = c_1^{\text{end}} + (c_1^{\text{start}} - c_1^{\text{end}}) \times \alpha$$

where $\alpha = 1 - \frac{\text{update}_i}{\text{total_updates}}$. We implemented this by creating a subclass of `PPOModel` and overriding a single `_current_value_coeff()` hook (implemented to simplify this approach in the first place); all other components remain identical to the base for better comparison.

3.4 PPO with an LSTM architecture

3.5 PPO with a replay buffer

4 Experimental Results

5 Conclusion and Future Work

6 Contributions and Acknowledgements

Contributions The code was separated in three key parts:

- Adrien took care of the benchmarking, base PPO implementation and value-function annealing implementation
- Eric took care of the LSTM agent and running the benchmarks
- Sumukh took care of implementing the replay buffer agent

The report was also split:

- Adrien wrote the Abstract, Introduction, and the parts of the methodology regarding the general code, base PPO implementation and value-function annealing implementation
- Eric wrote the part of the methodology regarding the LSTM agent and the Experimental Results
- Sumukh wrote the background, the part of the methodology regarding the replay buffer agent and the Conclusion

Acknowledgements a

References

Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphaël Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=nIAxjsniDzg`.

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei Rusu, Joel Veness, Marc Bellemare, Alex Graves, Martin Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–33, 02 2015. doi: 10.1038/nature14236.

Tianwei Ni, Benjamin Eysenbach, and Ruslan Salakhutdinov. Recurrent model-free rl can be a strong baseline for many pomdps. 2022. URL `https://arxiv.org/abs/2110.05038`.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL `http://arxiv.org/abs/1707.06347`.

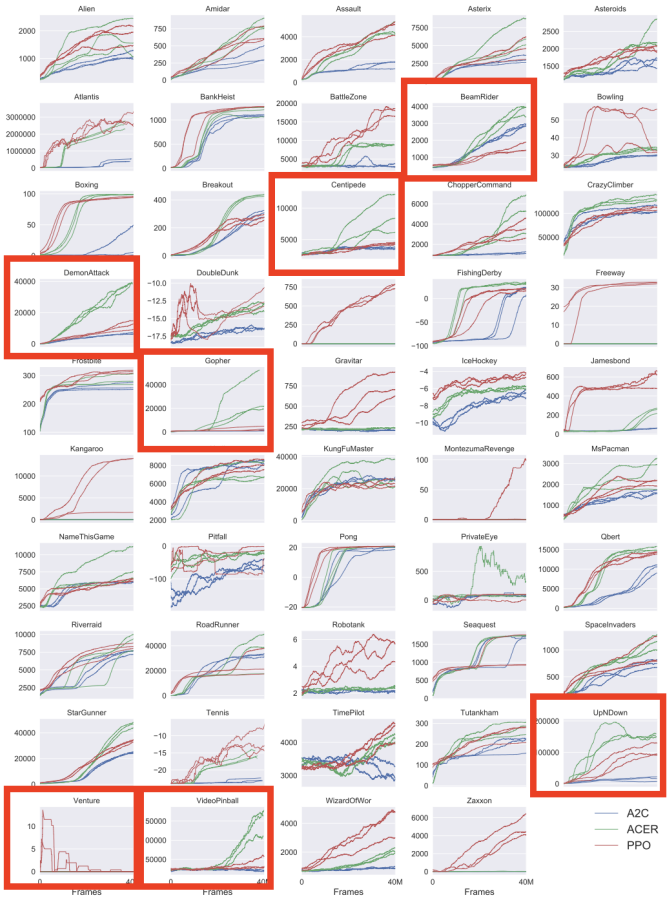
Ziyu Wang, Victor Bapst, Nicolas Heess, Volodymyr Mnih, Remi Munos, Koray Kavukcuoglu, and Nando de Freitas. Sample efficient actor-critic with experience replay. 2017. URL `https://arxiv.org/abs/1611.01224`.

Annex

	A2C	ACER	PPO
Alien	1141.7	1655.4	**1850.3**
Amidar	380.8	**827.6**	674.6
Assault	1562.9	4653.8	**4971.9**
Asterix	3176.3	**6801.2**	4532.5
Asteroids	1653.3	**2389.3**	2097.5
Atlantis	729265.3	1841376.0	**2311815.0**
BankHeist	1095.3	1177.5	**1280.6**
BattleZone	3080.0	8983.3	**17366.7**
BeamRider	3031.7	**3863.3**	1590.0
Bowling	30.1	33.3	**40.1**
Boxing	17.7	**98.9**	94.6
Breakout	303.0	**456.4**	274.8
Centipede	3496.5	**8904.8**	4386.4
ChopperCommand	1171.7	**5287.7**	3516.3
CrazyClimber	107770.0	**132461.0**	110202.0
DemonAttack	6639.1	**38808.3**	11378.4
DoubleDunk	-16.2	**-13.2**	-14.9
Enduro	0.0	0.0	**758.3**
FishingDerby	20.6	**34.7**	17.8
Freeway	0.0	0.0	**32.5**
Frostbite	261.8	285.6	**314.2**
Gopher	1500.9	**37802.3**	2932.9
Gravitar	194.0	225.3	**737.2**
IceHockey	-6.4	-5.9	**-4.2**
Jamesbond	52.3	261.8	**560.7**
Kangaroo	45.3	50.0	**9928.7**
Krull	**8367.4**	7268.4	7942.3
KungFuMaster	24900.3	**27599.3**	23310.3
MontezumaRevenge	0.0	0.3	**42.0**
MsPacman	1626.9	**2718.5**	2096.5
NameThisGame	5961.2	**8488.0**	6254.9
Pitfall	-55.0	**-16.9**	-32.9
Pong	19.7	**20.7**	20.7
PrivateEye	91.3	**182.0**	69.5
Qbert	10065.7	**15316.6**	14293.3
Riverraid	7653.5	**9125.1**	8393.6
RoadRunner	32810.0	**35466.0**	25076.0
Robotank	2.2	2.5	**5.5**
Seaquest	1714.3	**1739.5**	1204.5
SpaceInvaders	744.5	**1213.9**	942.5
StarGunner	26204.0	**49817.7**	32689.0
Tennis	-22.2	-17.6	**-14.8**
TimePilot	2898.0	4175.7	**4342.0**
Tutankham	206.8	**280.8**	254.4
UpNDown	17369.8	**145051.4**	95445.0
Venture	0.0	0.0	0.0
VideoPinball	19735.9	**156225.6**	37389.0
WizardOfWor	859.0	2308.3	**4185.3**
Zaxxon	16.3	29.0	**5008.7**

Table 6: Mean final scores (last 100 episodes) of PPO and A2C on Atari games after 40M game frames (10M timesteps).

Annex 1: Results from running ACER, A2C and PPO on the Atari 57 benchmarks, rewards over the last 100 episodes. Encircled in red are the selected games.



Annex 2: Results from running ACER, A2C and PPO on the Atari 57 benchmarks, presented cumulative rewards over training. Encircled in red are the selected games.

Hyperparameter	Value
Horizon (T)	128
Adam stepsize	$2.5 \times 10^{-4} \times \alpha$
Num. epochs	3
Minibatch size	32×8
Discount (γ)	0.99
GAE parameter (λ)	0.95
Number of actors	8
Clipping parameter ϵ	$0.1 \times \alpha$
VF coeff. c_1 (9)	1
Entropy coeff. c_2 (9)	0.01

Annex 3: Hyperparameters used in the original PPO paper and in our experiments.